

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Операционные системы»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Тесты производительности Linpack»

Выполнили:

Жук Анастасия Валерьевна

студентка группы N3248



Проверил:

Кича Игорь Владимирович

инженер факультета БИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2024 г.

Оглавление

Введение.....	4
Основная часть	5
Теория.....	5
Практика.....	6
Вывод	18
Список используемых источников	19

Введение

Все на одной ОС

Найти и скомпилировать программу `linpack` для оценки производительности компьютера (Flops) и протестировать ее при различных режимах работы ОС:

1. С различными приоритетами задачи в планировщике
2. С наличием и отсутствием привязки к процессору
3. Провести несколько тестов, сравнить результаты по 3 сигма или другим статистическим критериям

Изменить параметры на уровне ядра (выбрать одно):

- Повлиять на настройки имеющегося планировщика

Основная часть

Теория

Планировщик в операционной системе (ОС) — это компонент, отвечающий за распределение процессорного времени между всеми запущенными процессами и потоками. Он управляет порядком, в котором процессы и потоки получают доступ к центральному процессору (ЦП).

Основные функции планировщика ОС:

- Переключение контекста. Планировщик переключает контекст процессора с одного процесса или потока на другой.
- Приоритеты. Планировщик использует систему приоритетов для определения того, какие процессы должны выполняться в первую очередь.
- Алгоритмы планирования. Планировщик реализует алгоритмы планирования, которые определяют, как и когда процессы будут выполняться.
- Справедливость и эффективность. Планировщик стремится обеспечить справедливый доступ к ЦП для всех процессов и максимизировать эффективность использования процессорных ресурсов, минимизируя простои и время ожидания.
- Обработка прерываний и асинхронных событий. Планировщик также отвечает за реагирование на прерывания и асинхронные события, которые могут потребовать немедленного переключения контекста или изменения в плане выполнения процессов.

Linpack — это тест для оценки производительности параллельных систем. Он решает систему линейных алгебраических уравнений с числами двойной точности.

Работа теста включает две стадии:

1. На первой, на которую у современных процессоров уходит основная часть времени, матрица коэффициентов приводится к треугольному виду.
2. На второй стадии полученная система уравнений решается.

Производительность, измеренная эталонным тестом Linpack показывает количество операций над 64-битными числами с плавающей запятой (сложений и умножений), которые компьютер выполнял за секунду, соотношение, обозначаемое «FLOPS».

Продолжительность теста зависит от многих факторов — производительности процессоров, количества памяти на узел, количества MPI ранков и OMP тредов (если используется гибридная схема параллелизации) и других.

Практика

```
admin@Ubuntu:~$ sudo -i
[sudo] password for admin:
root@Ubuntu:~# cd linpack
root@Ubuntu:~/linpack# sudo docker build -t linpack ./
[+] Building 1.7s (9/9) FINISHED
=> [internal] load build definition from Dockerfile                                0.1s
=> => transferring dockerfile: 179B                                              0.0s
=> [internal] load metadata for docker.io/library/gcc:4.9.4                    1.1s
=> [internal] load .dockerignore                                                 0.1s
=> => transferring context: 2B                                                  0.0s
=> [internal] load build context                                                0.1s
=> => transferring context: 2.20kB                                             0.0s
=> [1/4] FROM docker.io/library/gcc:4.9.4@sha256:6356ef8b29cc3522527a85b6c58a28626744514bea87a10ff2bf67599a7474f 0.0s
=> CACHED [2/4] COPY . /usr/src/linpack                                         0.0s
=> CACHED [3/4] WORKDIR /usr/src/linpack                                         0.0s
=> CACHED [4/4] RUN gcc -o linpack -O3 -march=native -lm linpack.c             0.0s
=> exporting to image                                                           0.0s
=> => exporting layers                                                         0.0s
=> => writing image sha256:c7f3b352f97e7f665cfa6aa730f22f29135b60522afc019edbde173d93534495 0.0s
=> => naming to docker.io/library/linpack                                       0.0s
```

Рис. 1 – Создание docker image

```
root@Ubuntu:~/linpack# sudo docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:
```

Reps	Time(s)	DGEFA	DGESL	OVERHEAD	KFLOPS
1024	0.55	74.19%	3.30%	22.51%	3327103.216
2048	1.05	73.91%	3.04%	23.05%	3469373.826
4096	2.12	73.83%	3.28%	22.89%	3446347.587
8192	4.03	74.21%	3.21%	22.58%	3609106.559
16384	5.81	73.84%	3.17%	22.99%	5028195.763
32768	11.60	73.86%	3.12%	23.02%	5037949.330

Рис. 2 – Запуск теста без изменения приоритета задач и привязки к процессору

Таблица 1. 3 сигма для теста без изменения приоритета задач и привязки к процессору

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3327103,216	3986346,047	744657,6004	1752373,246	6220318,848
2048	3469373,826				
4096	3446347,587				
8192	3609106,559				
16384	5028195,763				
32768	5037949,33				

Изменение приоритета

В Linux любой процесс может иметь приоритет от -20 до +19. Linux для установки приоритетов использует программу nice. Десять – это приоритет по умолчанию. Чем выше значение, тем приоритет ниже.

```
root@Ubuntu:~/linpack# sudo nice -n -20 docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:
```

Reps	Time(s)	DGEFA	DGESL	OVERHEAD	KFLOPS
1024	0.51	73.91%	3.11%	22.98%	3589992.325
2048	1.09	73.58%	3.24%	23.18%	3359403.492
4096	2.27	73.93%	3.47%	22.61%	3205659.872
8192	3.83	73.98%	3.22%	22.81%	3802570.752
16384	6.17	74.36%	3.36%	22.29%	4692780.106
32768	11.54	73.81%	3.15%	23.05%	5066287.135

Рис. 3 – Запуск теста с приоритетом задачи -20

Таблица 2. 3 сигма для теста с приоритетом задачи -20

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3589992,325	3952782,28	689439,4073	1884464,059	6021100,502
2048	3359403,492				
4096	3205659,872				
8192	3802570,752				
16384	4692780,106				
32768	5066287,135				

```

root@Ubuntu:~/linpack# sudo nice -n 0 docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:

Reps Time(s) DGEFA  DGESL  OVERHEAD  KFLOPS
-----
1024  0.52  75.64%  3.27%  21.09%  3446822.730
2048  1.07  74.58%  3.19%  22.22%  3390590.863
4096  1.60  73.54%  2.93%  23.53%  4596030.242
8192  3.49  73.81%  3.02%  23.17%  4194552.122
16384 5.63  74.08%  3.03%  22.89%  5179559.036
32768 11.58 73.93%  3.19%  22.88%  5037878.266

```

Рис. 4 - Запуск теста с приоритетом задачи 0

Таблица 3. 3 сигма для теста с приоритетом задачи 0

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3446822,73	4307572,21	703732,1928	2196375,632	6418768,788
2048	3390590,863				
4096	4596030,242				
8192	4194552,122				
16384	5179559,036				
32768	5037878,266				

```

root@Ubuntu:~/linpack# sudo nice -n 1 docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:

Reps Time(s) DGEFA  DGESL  OVERHEAD  KFLOPS
-----
1024  0.52  73.27%  3.22%  23.52%  3522231.851
2048  1.07  74.52%  3.31%  22.17%  3373708.634
4096  2.15  74.13%  3.26%  22.61%  3376592.402
8192  4.13  73.75%  3.14%  23.11%  3546366.179
16384 5.08  73.50%  3.04%  23.46%  5787146.894
32768 11.63 73.88%  3.19%  22.93%  5019536.176

```

Рис. 5 - Запуск теста с приоритетом задачи 1

Таблица 4. 3 сигма для теста с приоритетом задачи 1

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3522231,851	4104263,689	947192,9133	1262684,95	6945842,429
2048	3373708,634				
4096	3376592,402				
8192	3546366,179				
16384	5787146,894				
32768	5019536,176				

```

root@Ubuntu:~/linpack# sudo nice -n 19 docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:

      Reps Time(s) DGEFA  DGE SL  OVERHEAD  KFLOPS
-----
    1024   0.54  73.53%   3.16%  23.31%  3426515.438
    2048   0.91  73.67%   3.30%  23.02%  4025013.225
    4096   1.73  72.69%   3.06%  24.25%  4287130.038
    8192   3.52  73.68%   3.05%  23.27%  4169825.135
   16384   4.99  73.15%   3.06%  23.80%  5917233.169
   32768  10.06  73.78%   3.11%  23.11%  5815255.875

```

Рис. 6 - Запуск теста с приоритетом задачи 19

Таблица 5. 3 сигма для теста с приоритетом задачи 19

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3426515,438	4606828,813	931153,7739	1813367,492	7400290,135
2048	4025013,225				
4096	4287130,038				
8192	4169825,135				
16384	5917233,169				
32768	5815255,875				

Вывод: если сравнивать производительность при разных приоритетах, то сказать, что производительность зависит от заданного приоритета, так как нельзя заметить общую тенденцию (увеличение или уменьшение) значений.

Привязка к процессору

Привязка к процессору (англ. processor affinity), или закрепление процессора, или привязка к кэшу, — технология, которая обеспечивает закрепление и открепление процесса или потока к конкретному ядру центрального процессора, центральному процессору или набору процессоров, так что процесс или поток будут выполняться только на указанном ядре, процессоре или процессорах, а не на любом процессоре многопроцессорной системы.

Привязку процессора можно рассматривать как модификацию алгоритма планирования центральной очереди задач в многопроцессорной операционной системе. Каждому элементу в очереди задач сопоставлен тег, задающие «родственные» ему процессоры.

```
root@Ubuntu:~/linpack# sudo taskset -c 0 docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:
```

	Reps	Time(s)	DGEFA	DGESL	OVERHEAD	KFLOPS
	1024	0.51	73.60%	3.16%	23.24%	3620482.956
	2048	1.07	74.33%	3.02%	22.65%	3382659.508
	4096	2.05	73.67%	3.39%	22.94%	3560605.869
	8192	3.75	74.01%	3.13%	22.86%	3892487.166
	16384	5.82	73.75%	3.05%	23.21%	5037707.384
	32768	11.63	73.72%	3.10%	23.18%	5037460.386

Рис. 7 - Запуск теста с привязкой к 1 процессору

Таблица 6. 3 сигма для теста с привязкой к 1 процессору

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3620482,956	4088567,212	687491,1446	2026093,778	6151040,645
2048	3382659,508				
4096	3560605,869				
8192	3892487,166				
16384	5037707,384				
32768	5037460,386				

```
root@Ubuntu:~/linpack# sudo taskset -c 2 docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:
```

Reps	Time(s)	DGEFA	DGESL	OVERHEAD	KFLOPS
1024	0.57	73.66%	3.29%	23.05%	3214414.151
2048	1.06	74.28%	2.88%	22.84%	3439921.758
4096	1.91	73.97%	3.09%	22.94%	3828174.588
8192	3.14	73.78%	2.93%	23.29%	4668429.965
16384	4.70	73.34%	2.90%	23.75%	6272729.114
32768	9.61	73.51%	2.97%	23.53%	6122954.203
65536	19.75	73.52%	2.99%	23.49%	5955099.230

Рис. 8 - Запуск теста с привязкой к 3 процессору

Таблица 7. 3 сигма для теста с привязкой к 3 процессору

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3214414,151	4785960,43	1229265,98	1098162,49	8473758,37
2048	3439921,758				
4096	3828174,588				
8192	4668429,965				
16384	6272729,114				
32768	6122954,203				
65536	5955099,23				

Вывод: при привязке к процессору производительность меняется, но на то, какое будет значение, влияет уже сам процессор (его производительность).

Изменение настроек планировщика

```
root@Ubuntu:~/linpack# sudo sysctl -A | grep "sched" | grep -v "domain"
kernel.sched_autogroup_enabled = 1
kernel.sched_cfs_bandwidth_slice_us = 5000
kernel.sched_deadline_period_max_us = 4194304
kernel.sched_deadline_period_min_us = 100
kernel.sched_rr_timeslice_ms = 100
kernel.sched_rt_period_us = 1000000
kernel.sched_rt_runtime_us = 950000
kernel.sched_schedstats = 0
kernel.sched_util_clamp_max = 1024
kernel.sched_util_clamp_min = 1024
kernel.sched_util_clamp_min_rt_default = 1024
net.mptcp.scheduler = default
```

Рис. 9 – Настройки планировщика

Параметры планировщика:

1. kernel.sched_autogroup_enabled = 1

- Включает автоматическую группировку процессов, что позволяет более справедливо распределять ресурсы между процессами, создаваемыми одним родительским процессом.

2. kernel.sched_cfs_bandwidth_slice_us = 5000

- Устанавливает максимальное время (в микросекундах), которое процесс может использовать в рамках планировщика CFS (Completely Fair Scheduler) при наличии ограничений по пропускной способности.

3. kernel.sched_deadline_period_max_us = 4194304

- Максимальный период (в микросекундах) для задач, использующих планировщик Deadline.

4. kernel.sched_deadline_period_min_us = 100

- Минимальный период (в микросекундах) для задач, использующих планировщик Deadline.

5. kernel.sched_rr_timeslice_ms = 100

- Устанавливает время среза (в миллисекундах) для планировщика реального времени (RR — Round Robin). Это максимальное время, в течение которого задача может выполняться до переключения на другую.

6. kernel.sched_rt_period_us = 1000000

- Устанавливает период (в микросекундах) для задач реального времени.

7. kernel.sched_rt_runtime_us = 950000

- Устанавливает время (в микросекундах), в течение которого задачи реального времени могут выполняться в рамках заданного периода.

8. kernel.sched_schedstats = 0

- Включает или отключает сбор статистики по планировщику. Если установлено в 1, статистика будет собираться.

9. `kernel.sched_util_clamp_max = 1024`

- Устанавливает максимальное значение для ограничения полезной нагрузки, используемое для управления приоритетом задач.

10. `kernel.sched_util_clamp_min = 1024`

- Устанавливает минимальное значение для ограничения полезной нагрузки.

11. `kernel.sched_util_clamp_min_rt_default = 1024`

- Устанавливает минимальное значение для ограничения полезной нагрузки по умолчанию для задач реального времени.

12. `net.mptcp.scheduler = default`

- Параметр, относящийся к многопоточному TCP (MPTCP). Указывает, что используется планировщик по умолчанию для MPTCP.

```
root@Ubuntu:~/linpack# sudo sysctl -w kernel.sched_rt_runtime_us=800000
kernel.sched_rt_runtime_us = 800000
root@Ubuntu:~/linpack# sudo sysctl -A | grep "sched" | grep -v "domain"
kernel.sched_autogroup_enabled = 1
kernel.sched_cfs_bandwidth_slice_us = 5000
kernel.sched_deadline_period_max_us = 4194304
kernel.sched_deadline_period_min_us = 100
kernel.sched_rr_timeslice_ms = 100
kernel.sched_rt_period_us = 1000000
kernel.sched_rt_runtime_us = 800000
kernel.sched_schedstats = 0
kernel.sched_util_clamp_max = 1024
kernel.sched_util_clamp_min = 1024
kernel.sched_util_clamp_min_rt_default = 1024
net.mptcp.scheduler = default
```

Рис. 10 - Изменением `kernel.sched_rt_runtime_us=800000`

```

root@Ubuntu:~/linpack# sudo docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:

```

Reps	Time(s)	DGEFA	DGESL	OVERHEAD	KFLOPS
1024	0.53	73.95%	3.09%	22.96%	3436336.772
2048	1.02	74.27%	3.35%	22.38%	3560980.036
4096	1.52	74.14%	3.20%	22.66%	4770229.035
8192	2.98	74.02%	3.19%	22.79%	4893774.296
16384	5.95	73.89%	3.24%	22.88%	4902734.559
32768	9.75	73.45%	2.95%	23.60%	6043241.778
65536	23.97	74.11%	3.22%	22.67%	4856577.980

Рис. 11 - Запуск теста с изменением kernel.sched_rt_runtime_us=800000

Таблица 8. 3 сигма для теста с изменением kernel.sched_rt_runtime_us=800000

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3436336,772	4637696,351	826302,9584	2158787,476	7116605,226
2048	3560980,036				
4096	4770229,035				
8192	4893774,296				
16384	4902734,559				
32768	6043241,778				
65536	4856577,98				

```

root@Ubuntu:~/linpack# sudo sysctl -w kernel.sched_rt_runtime_us=1000000
kernel.sched_rt_runtime_us = 1000000
root@Ubuntu:~/linpack# sudo sysctl -A | grep "sched" | grep -v "domain"
kernel.sched_autogroup_enabled = 1
kernel.sched_cfs_bandwidth_slice_us = 5000
kernel.sched_deadline_period_max_us = 4194304
kernel.sched_deadline_period_min_us = 100
kernel.sched_rr_timeslice_ms = 100
kernel.sched_rt_period_us = 1000000
kernel.sched_rt_runtime_us = 1000000
kernel.sched_schedstats = 0
kernel.sched_util_clamp_max = 1024
kernel.sched_util_clamp_min = 1024
kernel.sched_util_clamp_min_rt_default = 1024
net.mptcp.scheduler = default

```

Рис. 12 - Изменением kernel.sched_rt_runtime_us=1000000

```

root@Ubuntu:~/linpack# sudo docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:

Reps Time(s) DGEFA DGESL OVERHEAD KFLOPS
-----
1024 0.59 74.12% 3.58% 22.29% 3052752.838
2048 1.12 74.74% 3.47% 21.79% 3211940.017
4096 1.80 74.00% 3.32% 22.68% 4046238.145
8192 2.98 73.88% 3.20% 22.92% 4891072.293
16384 6.07 74.20% 3.25% 22.55% 4786443.621
32768 11.97 74.13% 3.26% 22.61% 4856850.540

```

Рис. 13 - Запуск теста с изменением kernel.sched_rt_runtime_us=1000000

Таблица 9. 3 сигма для теста с изменением kernel.sched_rt_runtime_us=1000000

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3052752,838	4140882,909	768988,7269	1833916,728	6447849,09
2048	3211940,017				
4096	4046238,145				
8192	4891072,293				
16384	4786443,621				
32768	4856850,54				

Вывод: при увеличении значения kernel.sched_rt_runtime_us, производительность меняется в лучшую сторону (ускорение процессов), а при уменьшении – в худшую (время увеличивается).

```

root@Ubuntu:~/linpack# sudo sysctl -w kernel.sched_rr_timeslice_ms=50
kernel.sched_rr_timeslice_ms = 50
root@Ubuntu:~/linpack# sudo sysctl -A | grep "sched" | grep -v "domain"
kernel.sched_autogroup_enabled = 1
kernel.sched_cfs_bandwidth_slice_us = 5000
kernel.sched_deadline_period_max_us = 4194304
kernel.sched_deadline_period_min_us = 100
kernel.sched_rr_timeslice_ms = 50
kernel.sched_rt_period_us = 1000000
kernel.sched_rt_runtime_us = 950000
kernel.sched_schedstats = 0
kernel.sched_util_clamp_max = 1024
kernel.sched_util_clamp_min = 1024
kernel.sched_util_clamp_min_rt_default = 1024
net.mptcp.scheduler = default

```

Рис. 14 - Изменение kernel.sched_rr_timeslice_ms=50

```

root@Ubuntu:~/linpack# sudo docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:

  Reps Time(s) DGEFA  DGESL  OVERHEAD  KFLOPS
-----
  1024   0.51  73.59%   3.50%  22.92%  3579083.104
  2048   0.64  73.57%   3.08%  23.36%  5723048.916
  4096   1.20  73.68%   3.05%  23.27%  6101909.414
  8192   2.39  73.48%   3.04%  23.48%  6161467.461
 16384   5.69  74.02%   3.26%  22.71%  5114221.674
 32768  12.08  73.93%   3.32%  22.75%  4821706.468

```

Рис. 15 - Запуск теста с изменением kernel.sched_rr_timeslice_ms=50

Таблица 10. 3 сигма для теста с изменением kernel.sched_rr_timeslice_ms=50

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3579083,104	5250239,506	892013,8164	2574198,057	7926280,955
2048	5723048,916				
4096	6101909,414				
8192	6161467,461				
16384	5114221,674				
32768	4821706,468				

```

root@Ubuntu:~/linpack# sudo sysctl -w kernel.sched_rr_timeslice_ms=200
kernel.sched_rr_timeslice_ms = 200
root@Ubuntu:~/linpack# sudo sysctl -A | grep "sched" | grep -v "domain"
kernel.sched_autogroup_enabled = 1
kernel.sched_cfs_bandwidth_slice_us = 5000
kernel.sched_deadline_period_max_us = 4194304
kernel.sched_deadline_period_min_us = 100
kernel.sched_rr_timeslice_ms = 200
kernel.sched_rt_period_us = 1000000
kernel.sched_rt_runtime_us = 950000
kernel.sched_schedstats = 0
kernel.sched_util_clamp_max = 1024
kernel.sched_util_clamp_min = 1024
kernel.sched_util_clamp_min_rt_default = 1024
net.mptcp.scheduler = default

```

Рис. 16 - Изменение kernel.sched_rr_timeslice_ms=200


```

root@Ubuntu:~/linpack# sudo docker run -it --rm linpack
Memory required: 315K.

LINPACK benchmark, Double precision.
Machine precision: 15 digits.
Array size 200 X 200.
Average rolled and unrolled performance:

      Reps Time(s) DGEFA  DGESL  OVERHEAD    KFLOPS
-----
    1024   0.59  74.20%   3.28%  22.52%  3086812.956
    2048   1.08  74.08%   3.24%  22.68%  3366839.083
    4096   1.85  74.65%   3.27%  22.08%  3900130.162
    8192   2.99  73.86%   3.22%  22.92%  4889584.275
   16384   6.06  74.05%   3.18%  22.76%  4803613.686
   32768  11.06  73.83%   3.16%  23.01%  5283901.337

```

Рис. 17 - Запуск теста с изменением kernel.sched_rr_timeslice_ms=200

Таблица 11. 3 сигма для теста с изменением kernel.sched_rr_timeslice_ms=200

Reps	KFLOPS	Среднее	σ	Левая граница	Правая граница
1024	3086812,956	4221814,083	820074,284	1761591,231	6682036,935
2048	3366839,083				
4096	3900130,162				
8192	4889584,275				
16384	4803616,686				
32768	5283901,337				

Вывод: при уменьшении kernel.sched_rr_timeslice_ms производительность ухудшается, при увеличении – улучшается (но не сильно, по сути, производительность остается прежней, а значения FLOPS с погрешностью).

Анализ применения критерия 3 сигм

Все значения KFLOPS в наблюдаемых наборах данных находятся в пределах трёх сигм от среднего. Это свидетельствует о достаточно стабильной производительности программы при различных условиях. Поскольку все значения находятся в пределах трёх сигм, можно предположить, что распределение производительности близко к нормальному, без сильных "выбросов".

Вывод

В ходе выполнения работы была проанализирована работа Linpack при различных состояниях ОС, изучен принцип работы планировщика задач и история их развития в ОС Linux.

Список используемых источников

1. <https://github.com/ereyes01/linpack>
2. <https://garden.struchkov.dev/ru/dev/fundamental/%D0%9F%D0%BB%D0%B0%D0%BD%D0%B8%D1%80%D0%BE%D0%B2%D1%89%D0%B8%D0%BA-%D0%9E%D0%A1>
3. <https://sechenov-hpc.readthedocs.io/ru/latest/examples/hpl.html>
4. <https://habr.com/ru/articles/664116/>
5. <https://www.osp.ru/os/2002/09/181910>
6. <https://wiki.merionet.ru/articles/priority-processov-linux?ysclid=m4181k7gyd699939279>
7. <https://developer.ibm.com/tutorials/l-completely-fair-scheduler/>
8. https://csc.sibsutis.ru/sites/csc.sibsutis.ru/files/courses/os/Lecture_06.pdf
9. <https://www.ibm.com/developerworks/ru/library/l-scheduler/index.html>
10. <https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html>
11. <https://serverfault.com/questions/948401/debugging-and-fine-tuning-the-linux-process-scheduler>
12. <https://developer.ibm.com/tutorials/l-completely-fair-scheduler/>
13. <http://www.brendangregg.com/perf.html#SchedulerAnalysis>
14. <https://doc.opensuse.org/documentation/leap/archive/42.1/tuning/html/book.sle.tuning/ch.a.tuning.taskscheduler.html>
15. <https://man7.org/linux/man-pages/man7/sched.7.html>